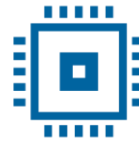




Transilvania
University
of Brasov



Transilvania
University
of Brasov

FACULTY OF ELECTRICAL ENGINEERING
AND COMPUTER SCIENCE

Department of Electronics and Computers

Study programme: Applied Electronics

Verification Project

Arithmetic Logic Unit

Table Of Contents

1	Digital circuit Project.....	3
1.1	Block Diagram.....	3
1.2	General Description of the DUT Functionality.....	3
1.3	DUT Interfaces	4
1.4	List of DUT Functionalities.....	6
1.5	Principle Schematic	7
1.6	Waveforms	8
1.7	Register Table	9
2	Verification of the Digital Circuit Project	13
2.1	Verification Environment Arhitecture.....	14
2.1.1	Coverage for the "IRQ" Interface	14
2.1.2	APB Driver	14
2.1.3	Monitor for the APB Interface.....	14
2.1.4	Monitor for the "IRQ" Interface.....	14
2.1.5	Monitor for the "result" interface.....	15
2.1.6	Transaction for the "IRQ" Interface	15
2.1.7	Transaction for the "result" Interface	15
2.2	List of Elements to Verify	15
2.3	Functional Coverage Elements.....	17
2.4	Tests.....	18
2.5	Obtained Results.....	19

Block Diagram

General Description of the DUT Functionality

DUT Interfaces

List of DUT Functionalities

Schematic Diagram

Waveforms

Register Table

1.1 BLOCK DIAGRAM

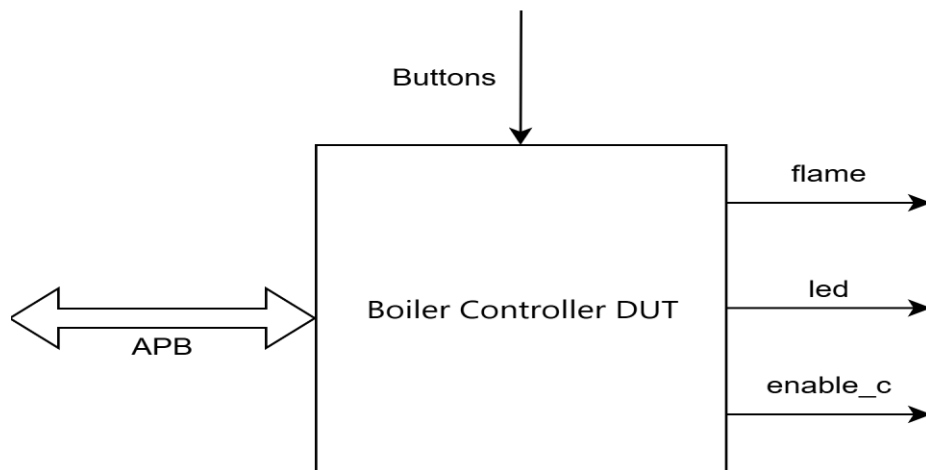


Figure 1. Block Diagram of the Design

1.2 GENERAL DESCRIPTION OF THE DUT FUNCTIONALITY

- Using a postfix expression evaluation algorithm, it performs basic arithmetic operations: addition, subtraction, multiplication, division, modulo, right shift, and left shift.
- The final result will be validated on the output interface by asserting the Valid signal.
- Certain situations (failed/completed transaction, FIFO empty/full, bit overflow of the result) will trigger an interrupt request.

Example:

- We will evaluate the infix expression $5 + ((1 + 2) * 4) - 3$, which can be written in postfix notation as $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$.

- Below is the sequence of steps through which the expression is evaluated according to the postfix evaluation algorithm.

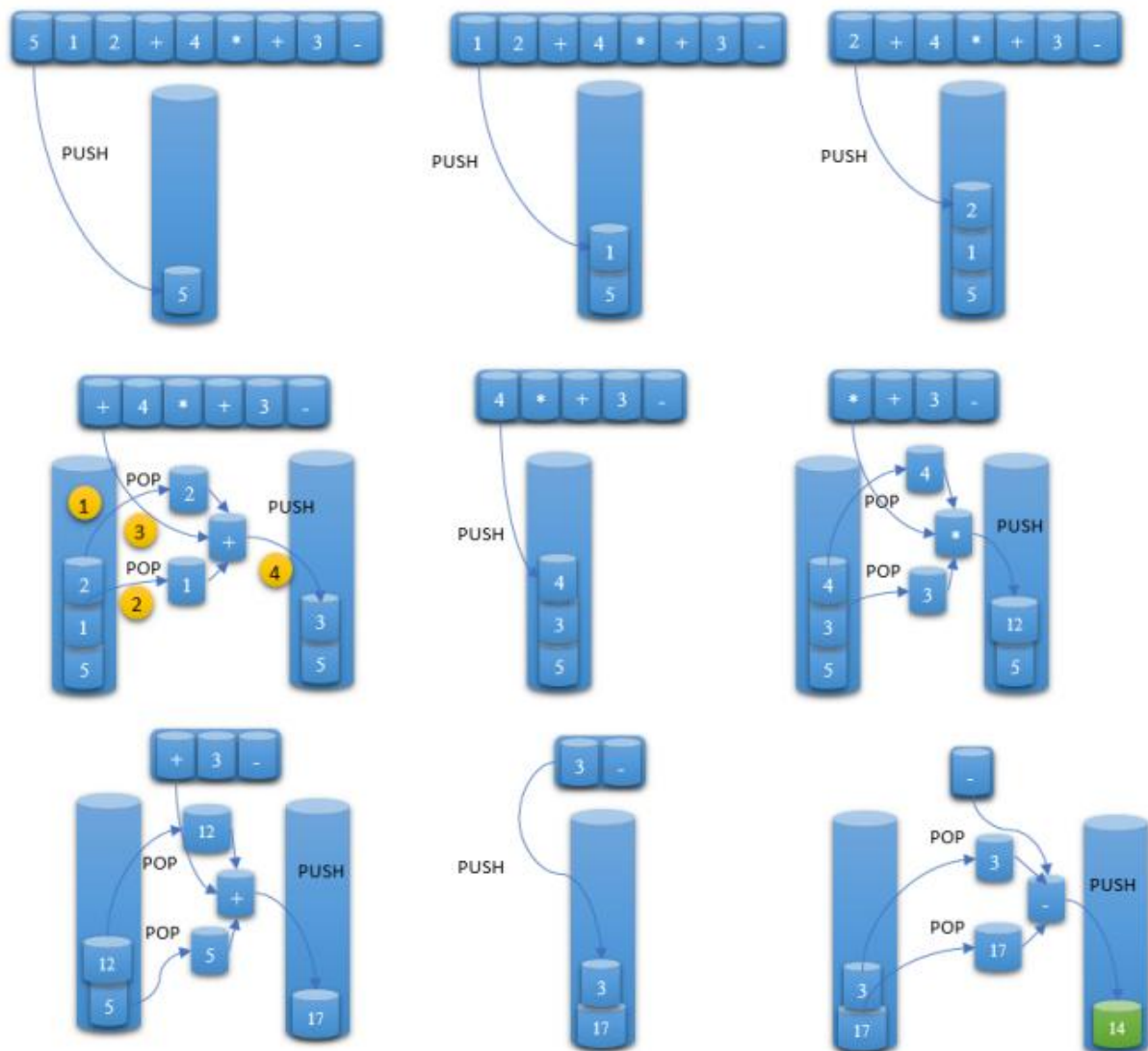


Figure 2. Evaluation of the expression $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$

1.3 DUT INTERFACES

System Interface			
Port Name	Size	Direction	Description

clk	1	input	System clock; used on the APB interface
rst_n	1	input	System reset; active low

APB Interface			
Port Name	Size	Direction	Description
psel	1	input	Select; indicates the start of the first transfer phase
penable	1	input	Enable; indicates the start of the second transfer phase
paddr	5	input	Address
pwrite	1	input	Write/read enable
pwdata	32	input	Write data
pready	1	output	Ready; indicates that the slave has completed the transfer
prdata	32	output	Read data

pslverr	1	output	Slave error; asserted along with pready if the slave encounters an error during transfer
---------	---	--------	--

Output Interface			
Port Name	Size	Direction	Description
result	16	output	Operation result
valid	1	output	Validation signal

Interrupt Interface			
Port Name	Size	Direction	Description
irq	1	output	Interrupt request

1.4 LIST OF DUT FUNCTIONALITIES

Functionality List:

- Performs basic arithmetic operations (addition, subtraction, multiplication, division, modulo, right shift, left shift);
- Provides an interrupt request feature;
- The interrupt request can be masked using the IRQ_MASK register;
- A configuration register allows the system to ignore one or more operations;

- A CLOCK_DIVIDER can be used to extend the duration of arithmetic operations;
- A control register allows clearing the interrupt register or the FIFO buffer storing data and operation types;
- If more than one bit is set in the OPERATIONS register in a single APB write, the pslverr signal is asserted;
- Invalid APB address access asserts the pslverr signal;
- To distinguish between data and operations in the FIFO buffer, an additional bit is added (0 for data, 1 for operations).

1.5 PRINCIPLE SCHEMATIC

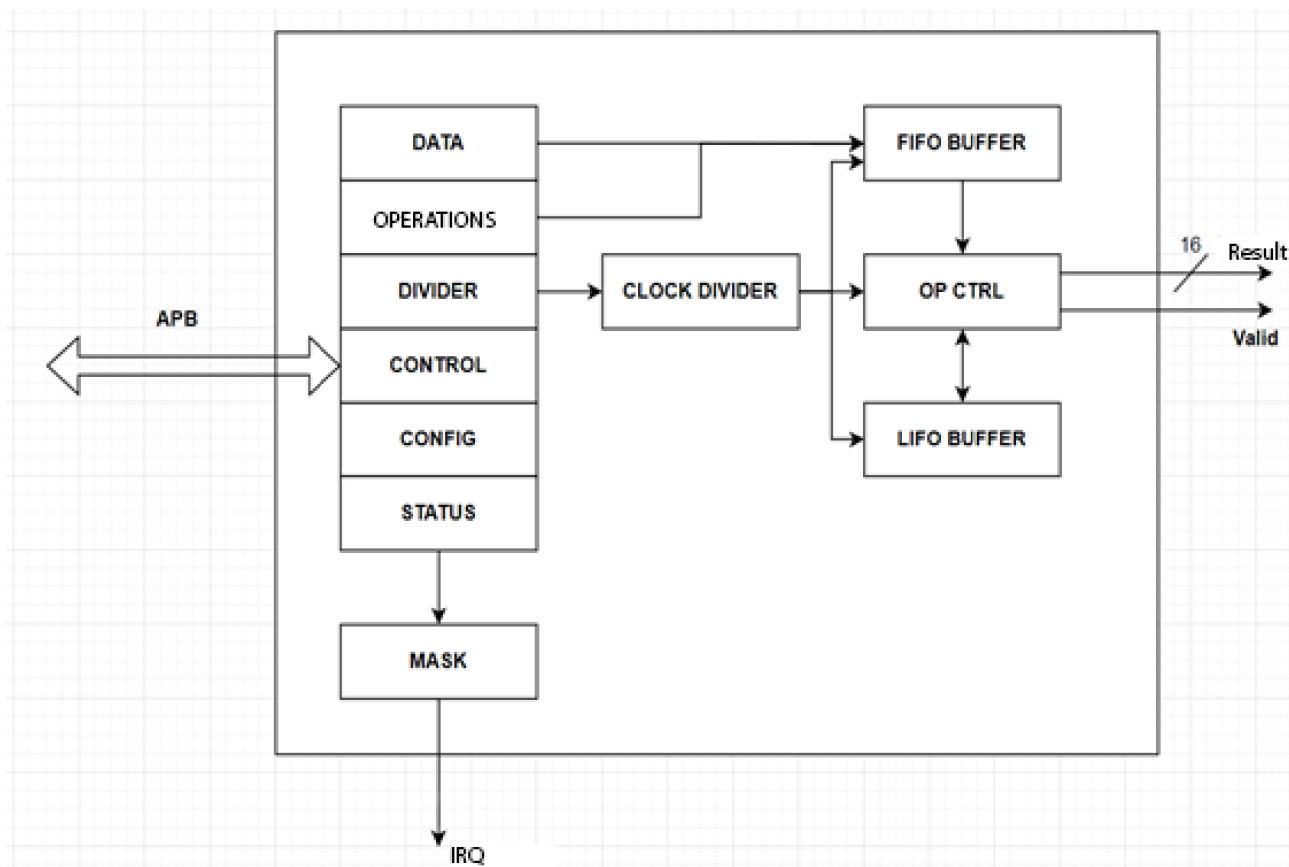


Figure 3. DUT Block Diagram

DUT Limitations:

- The FIFO Buffer can hold only 256 elements, meaning 128 data values and 127 operations.
- The number of data entries must always be equal to the number of operations + 1 for proper functionality.
- Before inserting data into the FIFO buffer, the CLOCK_DIVIDER must be set, and CLEAR_FIFO must be enabled in the CONTROL register;

- To ensure correct operation, only one bit should be written per transaction in the OPERATIONS register; otherwise, the DUT will signal an error on the APB interface by asserting pslverr;
- Data and operations must be entered in a specific order to ensure that, when extracted from the FIFO buffer and placed into the LIFO buffer, at least two data elements are always present in the LIFO.

1.6 WAVEFORMS

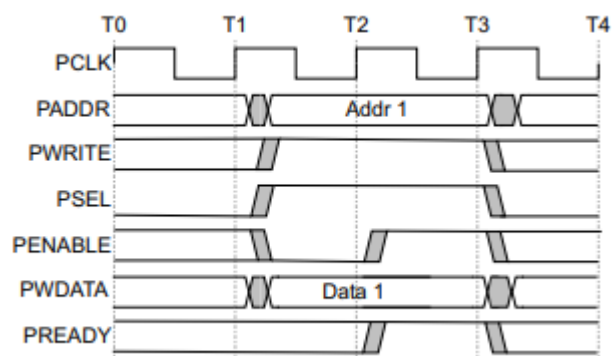


Figure 4. APB Write Transfer

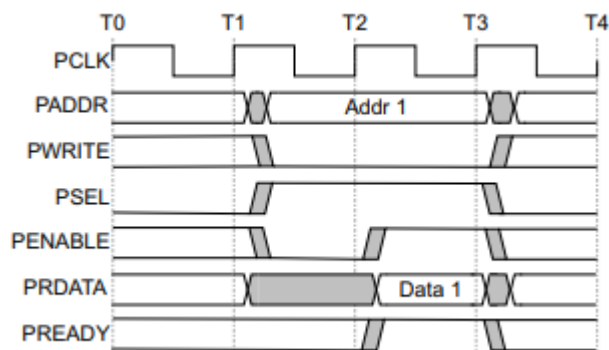


Figure 5. APB Read Transfer

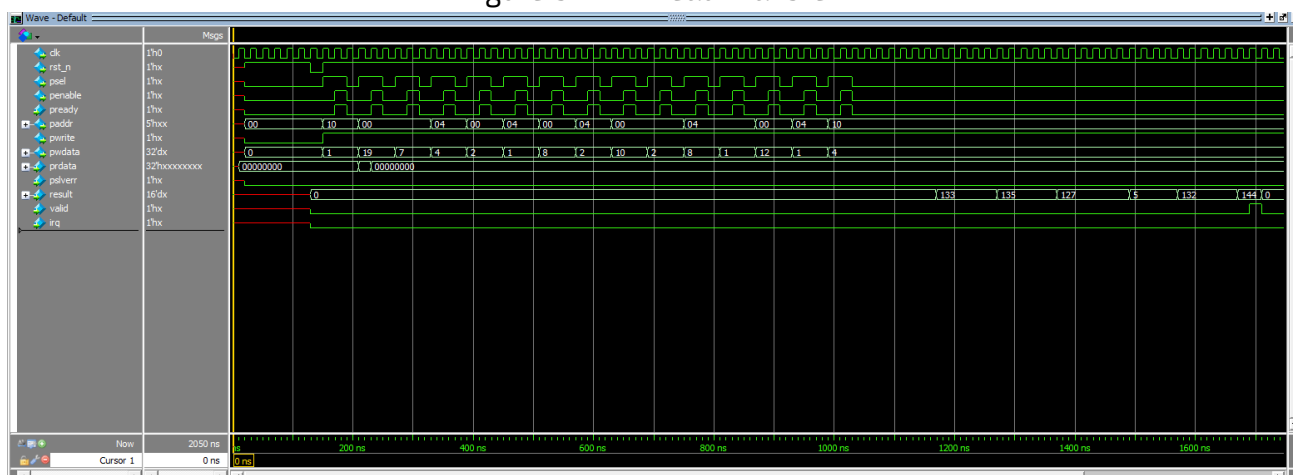


Figure 6. DUT Operational Waveforms

1.7 REGISTER TABLE

Registers Map:

Name	Address	Type	Reset value	Description
DATA	0x00	WO	8'b0	Register for input data
OPERATIONS	0x04	WO	8'b0	Register for input operations
STATUS	0x08	RO	8'b0	Transaction status register
DIVIDER	0x0C	RW	8'bFF	Clock divider register
CONTROL	0x10	RW	8'b0	Control register
MASK	0x14	RW	8'b0	Interrupt request mask register
CONFIG	0x18	RW	8'b0	Configuration register

DATA register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7:0	Value	WO	8'b0	Data for the operation

OPERATIONS Register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7	Reserved			
6	Shift Right	WO	1'b0	-
5	Shift Left	WO	1'b0	-
4	Modulo	WO	1'b0	-
3	Division	WO	1'b0	-
2	Multiplication	WO	1'b0	-
1	Subtraction	WO	1'b0	-
0	Addition	WO	1'b0	-

Status register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7:6	Reserved			
5	BSY	RO	1'b0	0: DUT is idle, no operation in progress

				1: An operation is being executed
4	Overflow	RO	1'b0	Indicates if the result exceeds the maximum bit width
3	Empty	RO	1'b0	Indicates if the FIFO buffer is empty
2	Full	RO	1'b0	Indicates if the FIFO buffer is full
1	Fail	RO	1'b0	Indicates if the operation failed
0	Done	RO	1'b0	Indicates successful execution of an operation

Divider register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7:0	CLOCK_DIV	RW	8'hFF	Clock divider value

Control register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7:4	Reserved			
3	Stop	RW	1'b0	Forces operation stop

2	Start	RW	1'b0	Initiates operation start
1	CLEAR_IRQ	RW	1'b0	Resets interrupt-triggering fields in the STATUS register
0	CLEAR_FIFO	RW	1'b0	Clears the FIFO buffer

MASK register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7:5	Reserved			
4	OVERFLOW_MASK	RW	1'b0	-
3	EMPTY_MASK	RW	1'b0	-
2	FULL_MASK	RW	1'b0	-
1	FAIL_MASK	RW	1'b0	-
0	DONE_MASK	RW	1'b0	-

CONFIG register

FIELD	NAME	TYPE	RESET VALUE	DESCRIPTION
7	Reserved			

6	IGNORE_SHIFT_RIGHT	RW	1'b0	If set, the shift right operation is refused
5	IGNORE_SHIFT_LEFT	RW	1'b0	If set, the shift left operation is refused
4	IGNORE_MOD	RW	1'b0	If set, the modulo operation is refused
3	IGNORE_DIV	RW	1'b0	If set, the division operation is refused
2	IGNORE_MUL	RW	1'b0	If set, the multiplication operation is refused
1	IGNORE_SUB	RW	1'b0	If set, the subtraction operation is refused
0	IGNORE_ADD	RW	1'b0	If set, the addition operation is refused

2 VERIFICATION OF THE DIGITAL CIRCUIT PROJECT

Verification Environment Architecture

List of elements to verify

Functional coverage elements

Tests

Obtained results

2.1 VERIFICATION ENVIRONMENT ARCHITECTURE

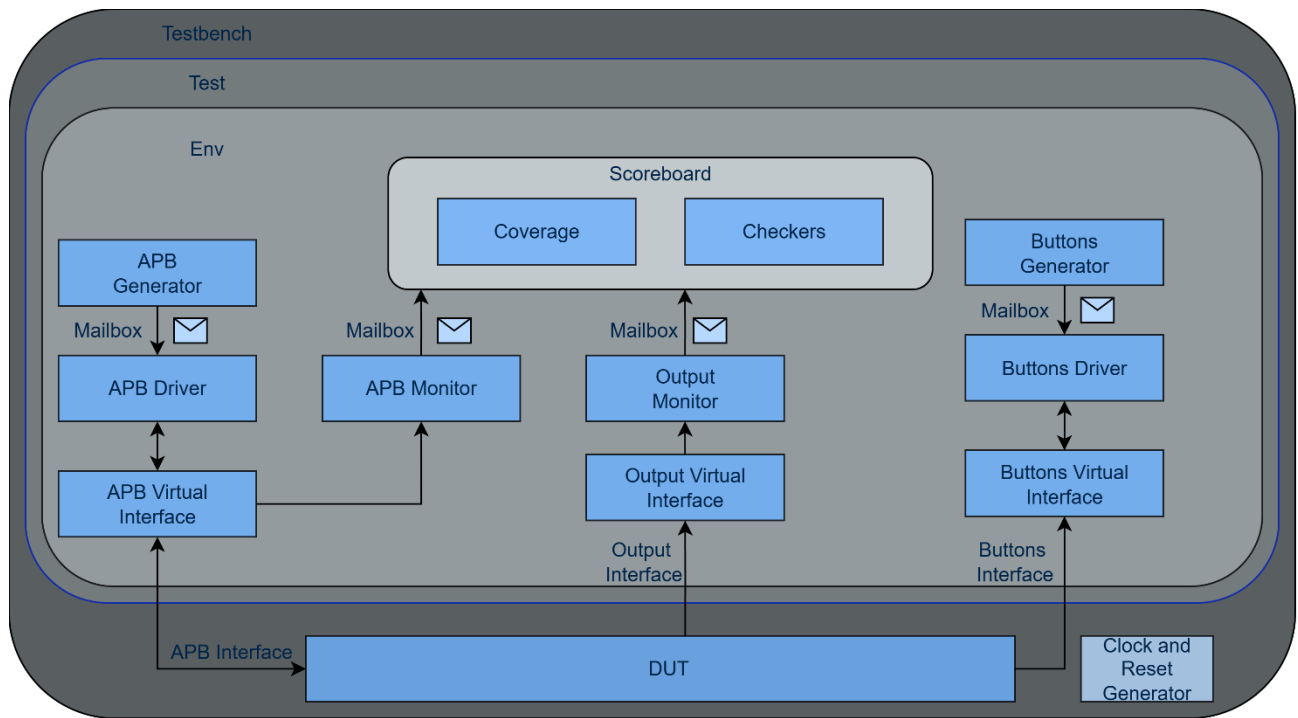


Figure 7. Overview of the verification environment

2.1.1 Coverage for the "IRQ" Interface

IRQ signal values are recorded. If the interrupt was only deactivated, the coverage value is 50%. If, during a test, the interrupt was both activated and deactivated, the coverage value will be 100%.

2.1.2 APB Driver

The driver takes transactions generated at the abstract level by the APB generator and sends them to the APB interface at the signal level.

2.1.3 Monitor for the APB Interface

Monitors the signals on the APB interface and sends transactions to the scoreboard.

2.1.4 Monitor for the "IRQ" Interface

This monitor implementation captures transactions (tracks signal activity) from the IRQ interface, creates corresponding transactions, sends them to the scoreboard for verification, and calls the functional coverage collector instance.

2.1.5 Monitor for the "result" interface

The monitor tracks traffic on the interface where the DUT places the results, retrieves data, and reconstructs transactions (in this case, as instances of the result_transaction class). The retrieved data is sent to the scoreboard for verification. The transaction is only captured when the validation signal is 1.

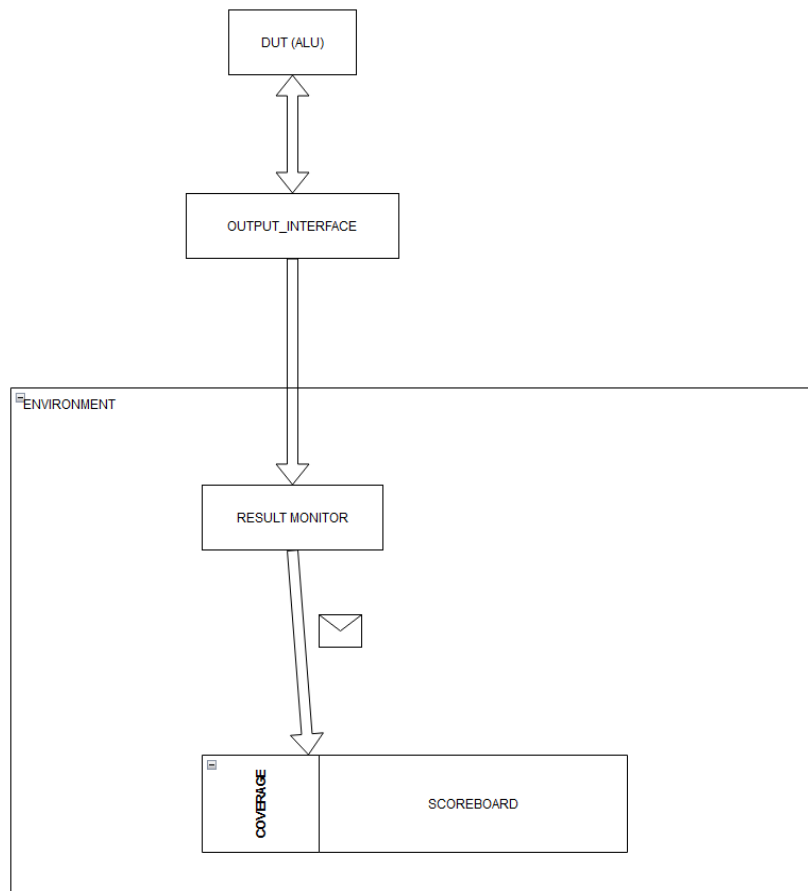


Figure 8. Logical diagram for "result"

2.1.6 Transaction for the "IRQ" Interface

This class represents an interrupt transaction, with a single irq attribute that indicates the state of the interrupt signal associated with the transaction. It also provides functions to display randomly generated values and create copies of transaction objects.

2.1.7 Transaction for the "result" Interface

This class contains a single 16-bit element that sequentially stores the results of operations performed in the DUT.

2.2 LIST OF ELEMENTS TO VERIFY

Assertion	Description
penable_signal_chk	The penable signal must have a concrete value during a

	transaction (i.e., when the psel signal is 1).
pwrite_signal_chk	The pwrite signal must have a concrete value during a transaction (i.e., when the psel signal is 1).
paddr_signal_chk	The paddr signal must have a concrete value during a transaction (i.e., when the psel signal is 1).
pready_signal_chk	The pready signal must have a concrete value during a transaction (i.e., when the psel signal is 1).
pslverr_signal_chk	The pslverr signal must have a concrete value during a transaction (i.e., when the psel signal is 1).
pwdata_signal_chk	The pwdata signal must have a concrete value during a transaction (i.e., when the psel signal is 1).
concurrent_pready_psel_chk	The pready signal must not assert simultaneously with psel.
concurrent_penable_psel_chk	The penable signal must not assert simultaneously with psel.
paddr_stable_chk	The paddr signal must remain stable (maintain its value) during a transaction (i.e., when the psel signal is 1).
pwrite_stable_chk	The pwrite signal must remain stable (maintain its value) during a transaction (i.e., when the psel signal is 1).
pwdata_stable_chk	The pwdata signal must remain stable (maintain its value) during a transaction (i.e., when the psel signal is 1).
pslverr_pready_sync_chk	The pslverr signal must assert simultaneously with pready.
penable_chk	The penable signal asserts on the next clock cycle after the psel signal has been asserted.
pready_chk	The pready signal must always come from the APB Slave (within a specific time frame).

2.3 FUNCTIONAL COVERAGE ELEMENTS

Event: cover_trans_done_ev - emitted at the end of a burst transaction	
Cover Point	Description
wr_enable	This coverpoint tracks whether the APB transaction is a write operation. It records all possible values of the write signal, providing an assessment of how often write transactions are generated compared to read transactions.
address	This coverpoint monitors the APB transaction address. It uses named bins for each valid ALU register address: DATA (0x00), OPS (0x04), STATUS (0x08), DIVIDER (0x0C), CONTROL (0x10), MASK (0x14), CONFIG (0x18). All other addresses are captured in a default "invalid" bin.
addr_x_rw	Cross coverage between the address coverpoint and the wr_enable coverpoint. It tracks which registers have been accessed by read transactions and which by write transactions.
write_data	This coverpoint tracks the data values written during APB write transactions. The value range is divided into meaningful bins: zero {0}, low {[1:15]}, mid {[16:255]}, hi {[256:65535]}, and very_large {[65536:\$]}.

option.per_instance = 1	If there are multiple instances for which coverage is calculated, we need to determine the value separately for each one.
-------------------------	---

Transaction recording from the result interface is performed in the result_monitor by calling the sample function.	
Cover point	Description
read_data	Applied to the result field in the result_transaction transaction, which can take values from 0 to 65535. The value range is divided into four bins: zero {0}, low {[1:15]}, mid {[16:255]}, and hi {[256:65535]}.

Transaction recording from the IRQ interface is performed in the IRQ_monitor by calling the sample function.	
Cover point	Description
Irq_cp	Measures coverage for the interrupt signal and indicates how many possible values of the signal have been sampled during simulation.

2.4 TESTS

Test File Name	Test Description
test_initial_t1	Directed test with 15 scenarios covering all ALU functionality: (1) complex postfix expression using ADD, SUB, MUL; (2) division; (3) modulo; (4) shift left; (5) shift right; (6) IRQ activation with DONE mask; (7) overflow detection (result > 65535); (8) division by zero producing FAIL; (9) pslverr on invalid/unaligned addresses; (10) CONFIG register blocking MUL operation; (11) register coverage sweep reading/writing all valid registers; (12) result = 0 via subtraction; (13)

	result in mid-range via addition; (14) result in large range via multiplication; (15) write data coverage with values in [256:65535] and [65536:\$] ranges.
--	---

2.5 OBTAINED RESULTS

Running the test_initial_t1 test produced the following results:

- APB functional coverage: 98.21% overall
- Address coverage: 100.00% (all 7 valid register bins hit: DATA, OPS, STATUS, DIVIDER, CONTROL, MASK, CONFIG, plus invalid addresses captured via default bin)
- Write data coverage: 100.00% (all 5 bins hit: zero, low, mid, hi, very_large)
- Read/write enable coverage: 100.00% (both read and write transactions sampled)
- Address × read/write cross coverage: 92.86% (13 out of 14 combinations; write-to-STATUS not expected as it is a read-only register)

Result functional coverage: 100.00% (all 4 bins hit: zero, low, mid, hi)

Interrupt (IRQ) functional coverage: 100.00% (both active and inactive states sampled)

APB transactions captured: 103 out of 103

ALU operations verified: 12 out of 12 (all results matched expected values)

Error conditions verified: 3 out of 3 (division by zero, CONFIG-blocked MUL, overflow)

Protocol error checks: 2 out of 2 (pslverr asserted for invalid and unaligned addresses)

IRQ behavior verified: IRQ correctly asserted when DONE_MASK enabled, cleared after CLEAR_IRQ

SVA protocol assertions: all 14 passed with no violations throughout the simulation